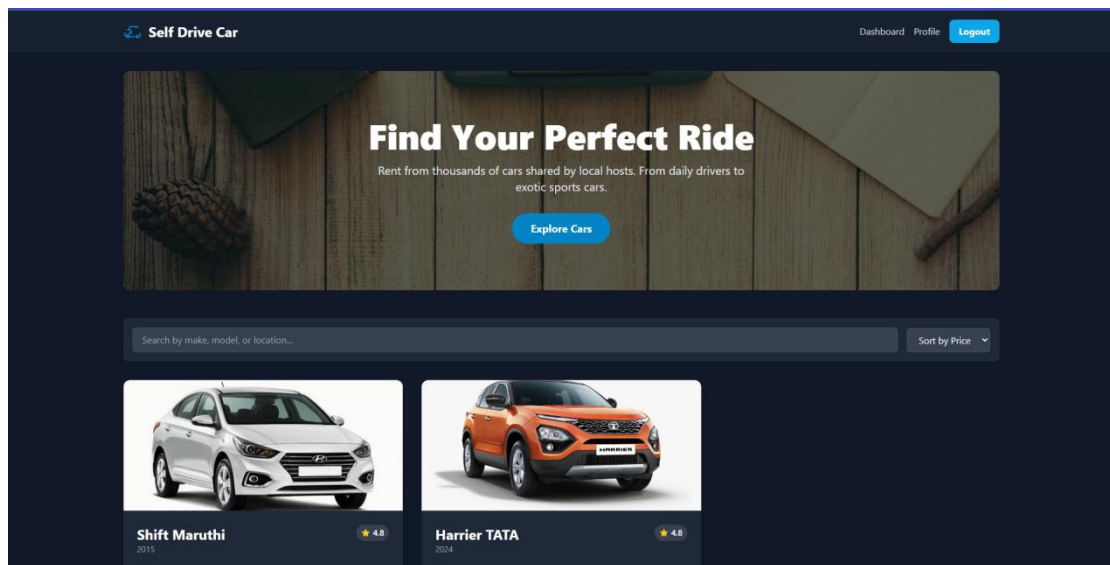


CHAPTER 6

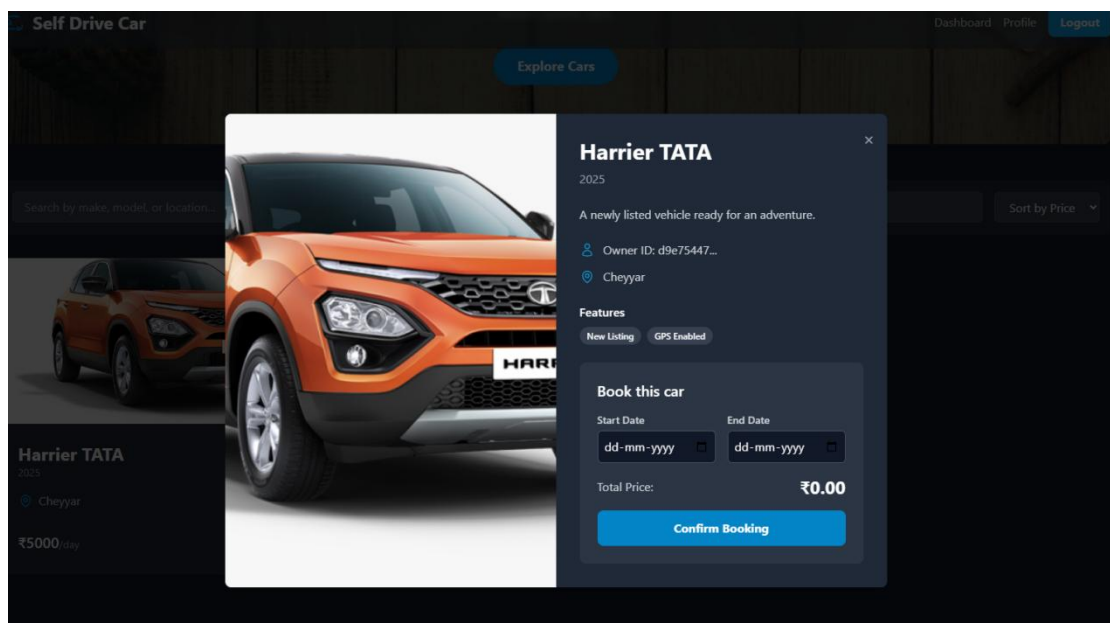
RESULT, DISCUSSION AND CONCLUSION

6.1 TEST REPORTS:

6.1.1 LANDING PAGE:



6.1.2 BOOKING SECTION:



6.1.3 LOGIN PAGE:

Self Drive Car

Login / Sign Up

Welcome Back

Sign in to continue

Email Address

Password

Sign In

Don't have an account? [Sign up](#)

© 2025 Self Drive Car. All rights reserved.

Your premier platform for peer-to-peer car rentals.

6.1.4 SIGNUP PAGE:

Self Drive Car

Login / Sign Up

Create Account

Get started with Self Drive Car

Full Name

Email Address

Password

Sign Up

Already have an account? [Sign in](#)

© 2025 Self Drive Car. All rights reserved.

Your premier platform for peer-to-peer car rentals.

6.1.5 USER DASHBOARD:

Self Drive Car

DashboardProfileLogout

Welcome, Kalaiselvi

List Your Car

Car Details

Make (e.g., Toyota)

Model (e.g., Camry)

2025

Location

Pricing & Image

5000

Click to upload or drag and drop

PNG, JPG, WEBP up to 5MB

List My Car Now

6.1.6 USER REGISTERD CAR SECTION:

My Cars for Rent



Shift Maruthi
₹2500/day



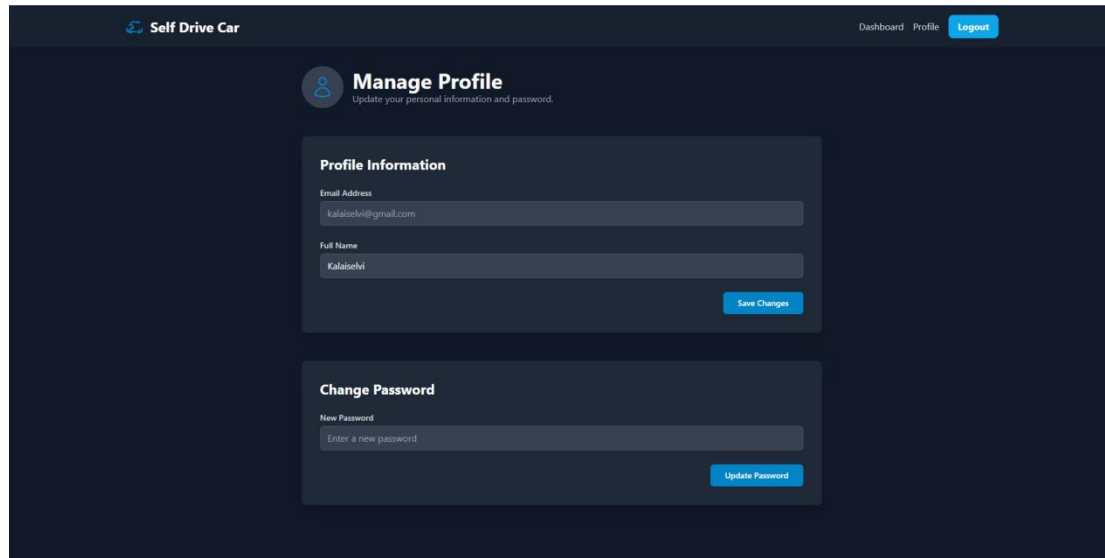
Harrier TATA
₹4998/day

6.1.7 USER BOOKING SECTION:

My Bookings

CAR ID	DATES	TOTAL PRICE	STATUS
27	2025-09-26 to 2025-09-30	₹19992.00	confirmed

6.1.8 USER PROFILE PAGE:



6.2 USER DOCUMENTATION:

LANDING PAGE:

```
import React, { useRef } from 'react';
import Hero from './Hero';
import CarList from './CarList';

const HomePage: React.FC = () => {
  const carListRef = useRef<HTMLDivElement>(null);

  const handleExploreClick = () => {
    carListRef.current?.scrollIntoView({ behavior: 'smooth' });
  };

  return (
    <div>
      <Hero onExploreClick={handleExploreClick} />
      <div ref={carListRef} id="car-list-section">
```

```

        <CarList />
      </div>
    </div>
  );
};

export default HomePage;

```

AUTHENTICATION:

```

// auth.ts
import { supabase } from "../supabaseClient";

// Sign up
export async function signUp(email: string, password: string, role: string) {
  const { data, error } = await supabase.auth.signUp({
    email,
    password,
    options: { data: { role } },
  });
  if (error) throw error;
  return data;
}

// Sign in
export async function signIn(email: string, password: string) {
  const { data, error } = await supabase.auth.signInWithPassword({
    email,
    password,
  });
  if (error) throw error;
  return data;
}

```

```
// Get current user
export async function getUser() {
  const { data, error } = await supabase.auth.getUser();
  if (error) throw error;
  return data?.user;
}
```

CAR LISTING MANAGEMENT:

```
// carService.ts
import React, { useState, useEffect } from 'react';
import { supabase } from '../services/supabaseClient';
import type { Car } from '../types';
import CarCard from './CarCard';
import CarDetailsModal from './CarDetailsModal';
import FilterSort from './FilterSort';

const CarList: React.FC = () => {
  const [cars, setCars] = useState<Car[]>([]); // Master list of available cars
  const [filteredCars, setFilteredCars] = useState<Car[]>([]); // List to be displayed
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [selectedCar, setSelectedCar] = useState<Car | null>(null);
  const [searchTerm, setSearchTerm] = useState("");
  const [sortOption, setSortOption] = useState("");

  useEffect(() => {
    const fetchCars = async () => {
      setLoading(true);
      const { data, error } = await supabase.from('cars').select('*');
      if (error) {
        setError(error.message);
      } else {
        // Pre-filter for available cars on initial fetch

```

```

        const availableCars = data?.filter(c => c.availability) || [];
        setCars(availableCars);
        setFilteredCars(availableCars);
    }
    setLoading(false);
};
fetchCars();
}, []);

```

// Effect to apply filters and sorting whenever dependencies change

```

useEffect(() => {
    let processedCars = [...cars];

    // Apply search filter
    if (searchTerm) {
        processedCars = processedCars.filter(car =>
            car.make.toLowerCase().includes(searchTerm.toLowerCase()) ||
            car.model.toLowerCase().includes(searchTerm.toLowerCase()) ||
            car.location.toLowerCase().includes(searchTerm.toLowerCase())
        );
    }

    // Apply sorting
    if (sortOption === 'price-asc') {
        processedCars.sort((a, b) => a.price_per_day - b.price_per_day);
    } else if (sortOption === 'price-desc') {
        processedCars.sort((a, b) => b.price_per_day - a.price_per_day);
    }

    setFilteredCars(processedCars);

}, [searchTerm, sortOption, cars]);

if (loading) {

```

```

    return <div className="text-center p-8">Loading cars...</div>;
  }
  if (error) {
    return <div className="text-center p-8 text-red-400">Error: {error}</div>;
  }
  return (
    <section>
      <FilterSort
        searchTerm={searchTerm}
        setSearchTerm={setSearchTerm}
        sortOption={sortOption}
        setSortOption={setSortOption}
      />
      {filteredCars.length > 0 ? (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
          {filteredCars.map(car => (
            <CarCard key={car.id} car={car} onSelectCar={setSelectedCar} />
          ))}
        </div>
      ) : (
        <div className="text-center py-16 px-4 bg-gray-800 rounded-lg">
          <h3 className="text-2xl font-bold text-white mb-2">No Cars Found</h3>
          <p className="text-gray-400">Try adjusting your search or filter settings to find
what you're looking for.</p>
        </div>
      )}
      {selectedCar && (
        <CarDetailsModal car={selectedCar} onClose={() => setSelectedCar(null)} />
      )}
    </section>
  );
};

export default CarList;

```

CAR BOOKING MANAGEMENT:

```
// bookingService.ts
import { supabase } from "../supabaseClient";

export async function createBooking(booking: {
  car_id: number;
  customer_id: number;
  start_date: string;
  end_date: string;
  total_amount: number;
  status: string;
}) {
  const { data, error } = await supabase.from("bookings").insert([booking]);
  if (error) throw error;
  return data;
}

export async function getBookingsByUser(customer_id: number) {
  const { data, error } = await supabase
    .from("bookings")
    .select("*", { count: "exact", orderBy: "daily_rate" })
    .eq("customer_id", customer_id);
  if (error) throw error;
  return data;
}
```

DASHBOARD:

```
const Dashboard: React.FC = () => {
  const { user } = useAuth();
  const [myCars, setMyCars] = useState<Car[]>([]);
  const [myBookings, setMyBookings] = useState<Booking[]>([]);
  const [initialLoading, setInitialLoading] = useState(true);
```

```

const [listsLoading, setListsLoading] = useState(false);

const fetchData = async () => {
  if (!user) return;
  setListsLoading(true);
  // Using Promise.all to fetch data in parallel for better performance
  const [carsResponse, bookingsResponse] = await Promise.all([
    supabase.from('cars').select('*').eq('owner_id', user.id),
    supabase.from('bookings').select('*').eq('renter_id', user.id)
  ]);

  if (carsResponse.error) console.error('Error fetching cars:', carsResponse.error);
  else setMyCars(carsResponse.data || []);

  if (bookingsResponse.error) console.error('Error fetching bookings:',
bookingsResponse.error);
  else setMyBookings(bookingsResponse.data || []);

  setListsLoading(false);
  if (initialLoading) {
    setInitialLoading(false);
  }
};

useEffect(() => {
  fetchData();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [user]);

if (initialLoading) return <p>Loading dashboard...</p>;

return (
  <div className="space-y-12">

```

```
<h1 className="text-4xl font-bold">Welcome, {user?.user_metadata?.full_name ||
user?.email}</h1>
```

```
<AddCarForm onCarAdded={fetchData} />
<div className="space-y-8">
  <div>
    <h2 className="text-3xl font-bold mb-4">My Cars for Rent</h2>
    {listsLoading ? <p className="text-gray-400">Refreshing list...</p> :
myCars.length > 0 ? (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {myCars.map(car => (
          <div key={car.id} className="bg-gray-800 p-4 rounded-lg">
            <img src={car.image_url} alt={car.model} className="w-full h-40
object-cover rounded-md mb-2"/>
            <h4 className="font-bold text-lg">{car.make} {car.model}</h4>
            <p>₹{car.price_per_day}/day</p>
          </div>
        ))}
      </div>
    ) : <p className="text-gray-400">You haven't listed any cars yet.</p>}
  </div>
```

```
<div>
  <h2 className="text-3xl font-bold mb-4">My Bookings</h2>
  {listsLoading ? <p className="text-gray-400">Refreshing list...</p> :
myBookings.length > 0 ? (
    <div className="bg-gray-800 rounded-lg overflow-hidden">
      <table className="min-w-full">
        <thead className="bg-gray-700">
          <tr>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider">Car ID</th>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider">Dates</th>
```

```

        <th className="px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider">Total Price</th>
        <th className="px-6 py-3 text-left text-xs font-medium text-gray-300
uppercase tracking-wider">Status</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-gray-700">
      {myBookings.map(booking => (
        <tr key={booking.id}>
          <td className="px-6 py-4 whitespace-
nowrap">{booking.car_id}</td>
          <td className="px-6 py-4 whitespace-
nowrap">{booking.start_date} to {booking.end_date}</td>
          <td className="px-6 py-4 whitespace-
nowrap">₹{booking.total_price.toFixed(2)}</td>
          <td className="px-6 py-4 whitespace-nowrap"><span
className="px-2 inline-flex text-xs leading-5 font-semibold rounded-full bg-green-800 text-
green-100">{booking.status}</span></td>
        </tr>
      )
    )}&;
  </tbody>
</table>
</div>
) : <p className="text-gray-400">You have no bookings.</p>
</div>
</div>
</div>
);
};
export default Dashboard;

```

USER PROFILE PAGE:

```
// Profile.tsx

import React, { useState, useEffect } from 'react';
import { useAuth } from '../hooks/useAuth';
import { UserIcon } from './icons/UserIcon';

const Profile: React.FC = () => {
  const { user, updateUser } = useAuth();
  const [fullName, setFullName] = useState("");
  const [password, setPassword] = useState("");
  const [loadingProfile, setLoadingProfile] = useState(false);
  const [loadingPassword, setLoadingPassword] = useState(false);
  const [profileMessage, setProfileMessage] = useState("");
  const [passwordMessage, setPasswordMessage] = useState("");
  const [profileError, setProfileError] = useState("");
  const [passwordError, setPasswordError] = useState("");

  useEffect(() => {
    if (user?.user_metadata?.full_name) {
      setFullName(user.user_metadata.full_name);
    }
  }, [user]);

  const handleProfileUpdate = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoadingProfile(true);
    setProfileMessage("");
    setProfileError("");

    const { error } = await updateUser({ fullName });

    if (error) {
      setProfileError(error.message);
    }
  }
}
```

```

    } else {
      setProfileMessage('Profile updated successfully!');
    }
    setLoadingProfile(false);
  };

const handlePasswordUpdate = async (e: React.FormEvent) => {
  e.preventDefault();
  if (password.length < 6) {
    setPasswordError('Password must be at least 6 characters long. ');
    return;
  }

  setLoadingPassword(true);
  setPasswordMessage('');
  setPasswordError('');

  const { error } = await updateUser({ password });

  if (error) {
    setPasswordError(error.message);
  } else {
    setPasswordMessage('Password updated successfully!');
    setPassword(''); // Clear password field
  }
  setLoadingPassword(false);
};

if (!user) {
  return <p>Loading user profile...</p>;
}

return (
  <div className="max-w-4xl mx-auto space-y-12">

```

```

<div className="flex items-center space-x-4">
  <div className="bg-gray-700 p-4 rounded-full">
    <UserIcon className="w-10 h-10 text-brand-primary" />
  </div>
  <div>
    <h1 className="text-4xl font-bold">Manage Profile</h1>
    <p className="text-gray-400">Update your personal information and
password.</p>
  </div>
</div>

{/* Profile Information Form */}
<div className="bg-gray-800 rounded-lg shadow-xl p-8">
  <h2 className="text-2xl font-bold text-white mb-6">Profile Information</h2>
  {profileError && <p className="bg-red-900/50 text-red-300 p-3 rounded-md mb-4
text-center">{profileError}</p>}
  {profileMessage && <p className="bg-green-900/50 text-green-300 p-3 rounded-
md mb-4 text-center">{profileMessage}</p>}
  <form onSubmit={handleProfileUpdate} className="space-y-6">
    <div>
      <label htmlFor="email" className="block text-sm font-medium text-gray-
300">Email Address</label>
      <input
        id="email"
        type="email"
        value={user.email || ""}
        disabled
        className="mt-1 block w-full bg-gray-700 border border-gray-600 rounded-
md shadow-sm py-2 px-3 text-gray-400 cursor-not-allowed"
      />
    </div>
    <div>
      <label htmlFor="fullName" className="block text-sm font-medium text-gray-
300">Full Name</label>

```

```

        <input
            id="fullName"
            type="text"
            value={fullName}
            onChange={(e) => setFullName(e.target.value)}
            required
            className="mt-1 block w-full bg-gray-700 border border-gray-600 rounded-
md shadow-sm py-2 px-3 text-white focus:outline-none focus:ring-brand-primary focus:border-
brand-primary"
        />
    </div>
    <div className="text-right">
        <button
            type="submit"
            disabled={loadingProfile}
            className="inline-flex justify-center py-2 px-6 border border-transparent
rounded-md shadow-sm text-sm font-medium text-white bg-brand-primary hover:bg-brand-
secondary focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-offset-gray-800
focus:ring-brand-secondary disabled:bg-gray-600"
            >
                {loadingProfile ? 'Saving...' : 'Save Changes'}
            </button>
    </div>
</form>
</div>
{ /* Change Password Form */ }
<div className="bg-gray-800 rounded-lg shadow-xl p-8">
    <h2 className="text-2xl font-bold text-white mb-6">Change Password</h2>
    {passwordError && <p className="bg-red-900/50 text-red-300 p-3 rounded-md
mb-4 text-center">{passwordError}</p>}
    {passwordMessage && <p className="bg-green-900/50 text-green-300 p-3
rounded-md mb-4 text-center">{passwordMessage}</p>}
    <form onSubmit={handlePasswordUpdate} className="space-y-6">
        <div>

```

```

        <label htmlFor="password" className="block text-sm font-medium text-gray-
300">New Password</label>
        <input
            id="password"
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            placeholder="Enter a new password"
            className="mt-1 block w-full bg-gray-700 border border-gray-600 rounded-
md shadow-sm py-2 px-3 text-white focus:outline-none focus:ring-brand-primary focus:border-
brand-primary"
        />
    </div>
    <div className="text-right">
        <button
            type="submit"
            disabled={loadingPassword}
            className="inline-flex justify-center py-2 px-6 border border-transparent
rounded-md shadow-sm text-sm font-medium text-white bg-brand-primary hover:bg-brand-
secondary focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-offset-gray-800
focus:ring-brand-secondary disabled:bg-gray-600"
        >
            {loadingPassword ? 'Updating...' : 'Update Password'}
        </button>
    </div>
</form>
</div>
</div>
);
};
export default Profile;

```

6.3 CONCLUSION:

The development of the Self-Drive Car Rental Platform successfully demonstrates the design and implementation of a scalable, secure, and user-friendly system for managing car rental services. The project integrates essential functionalities such as user authentication, car listing and management, booking and payment workflows, and profile handling. With role-based access control, both car owners and renters can seamlessly interact with the system in a transparent and efficient manner.

Through the implementation of a centralized dashboard and profile management, the system ensures improved usability, data accuracy, and streamlined operations. By leveraging Supabase for backend services, the platform benefits from real-time database operations, authentication handling, and secure data storage, reducing overhead and improving maintainability.

The results of testing confirm that the system meets its intended objectives by providing smooth navigation, accurate booking workflows, and effective error handling. While certain limitations still exist, such as dependency on internet connectivity and limited integration with external payment APIs, the system offers a solid foundation for future scalability and enhancements.

Overall, this project provides a practical solution to modern mobility challenges by enabling users to book self-drive cars easily and securely, while also offering car owners a convenient way to manage their fleets. It not only fulfills academic requirements but also highlights the real-world applicability of full-stack system design and deployment.

6.4 LIMITATIONS OF THE SYSTEM

Despite achieving its intended objectives, the Self-Drive Car Rental Platform has certain limitations:

- **Internet Dependency:** The system requires a stable internet connection for access, which may limit usage in areas with poor connectivity.
- **Payment Integration:** Currently, payment gateway support is basic or limited, restricting users to fewer options.

- **Scalability Constraints:** While the platform is designed to be modular, heavy traffic and large-scale usage may require advanced infrastructure optimization.
- **Device Support:** The system is primarily tested on web platforms; mobile application support is not yet fully optimized.
- **Limited Real-Time Tracking:** Vehicle tracking and GPS-based monitoring are not integrated, reducing transparency for both owners and renters.
- **Security Enhancements Needed:** Although standard authentication is provided, advanced security measures like multi-factor authentication and AI-driven fraud detection are not yet implemented.

6.5 FUTURE SCOPE OF THE PROJECT

To make the system more robust, scalable, and industry-ready, several enhancements can be considered in the future:

- **Mobile Application Development:** Launching Android and iOS apps to provide users with a seamless rental experience.
- **Advanced Payment Gateways:** Integration with multiple secure payment providers (UPI, PayPal, Stripe, etc.) for convenience.
- **GPS Tracking & Telematics:** Enabling real-time car location tracking, driver behavior monitoring, and route history for safety and transparency.
- **AI-Powered Recommendation Engine:** Suggesting cars to users based on their past bookings, preferences, and travel history.
- **Dynamic Pricing System:** Implementing surge pricing and seasonal discounts using data-driven algorithms.
- **Fleet Management Tools:** Providing car owners with advanced analytics on usage, revenue, and vehicle health monitoring.
- **Enhanced Security Features:** Adding biometric login, multi-factor authentication, and fraud detection systems.
- **International Expansion:** Supporting multiple currencies and languages to enable global deployment.